

Software Engineering in Industrial Practice (SEIP)

Dr. Ralf S. Engelschall

ENGINEERING FUNDAMENTALS

Software Engineering Metamodel

TECHNISCHE UNIVERSITÄT MÜNCHEN

EF 02.1

Intellectual Content: Version 1.0.4 (2021-09-16), Authored 2021 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.2 (2021-10-11), Copyright © 2021 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Craftsmanship Model 4

WHEREBY & HOW

Know how in a profession to operationally perform certain practices with the help of templates and tools.

Profession

A person's occupation, founded upon specialized educational training and formal qualification.

Know-How

Distinct overall knowledge on how to perform certain tasks.

Practice

Distinct steps to perform a certain task with the help of a certain know-how and template.

Template

The initial version of an artifact in order to speed up and/or support crafting the artifact.

Tool

Software application to create an artifact from scratch or from a template and further edit it.

Discipline Model 1

HOW & WHAT

Based on personal inclinations, understand how to act in area-grouped disciplines and create artifacts and their aspects.

Inclination

A person's natural tendency or urge to act or feel in a certain way.

Area

Logical group of strongly related disciplines.

Discipline

Distinct knowledge, responsibility and objectives within Software Engineering.

Artifact

Single representation of an internally used and/or externally delivered result.

Aspect

Smallest result unit of an artifact which can be created separately.

Workflow Model 2

WHAT & WHO

Understand the workflow of separate iterating cycles and work streams, comprised of one or more steps.

Workflow

The five distinct iteration cycles Product Management, Business, Development, Operations and Project Management.

Cycle

Logical iteration cycle of a sequence of strongly related and dependent workflow steps.

Step

Single step in a workflow cycle with a clear logical previous and next step in the logical work flow.

Flow

Logical sequence of interweaved, inter-cycle, optionally skipped, and dependent steps.

Key Message

Mastering Software Engineering means to understanding (in this order) **Discipline**, **Workflow**, **Process** and **Craftsmanship** models. Especially, *never* start with a particular Process model, as it is secondary!

Process Model 3

WHO & WHEN

Map activities onto a project execution schedule, based on horizontal team focus tracks and vertical temporal project periods and phases.

Role

Distinct responsibility and goal of an individual person.

Track

Work stream consisting of a continuously sequence of steps of a workflow cycle.

Activity

Distinct activity in the process to perform a certain task or complete a particular work-package.

Period

Minor time period in the process, to split the project phases into distinct time-, cost-, or scope-based time units.

Phase

Major time period in the process, to split the project into distinct focus periods, separated by major milestones.

suitable for

correlates to

serialized by 1

operationalised by

corresponds to

operationalises 1

derived from

big picture & coherency

ENGINEERING FUNDAMENTALS

Software Engineering Disciplines

TECHNISCHE UNIVERSITÄT MÜNCHEN

ANALYSIS AN

Software Requirements REQ

Identify Needs: 3 BB
We understand which outcomes of the solution are most valuable to users.

Requirements Engineer / Business Analyst

Domain Modeling DOM

Determine Solution: 2 WB
We model and specify the solution through involved functional and non-functional aspects.

Business Analyst / Business Architect

ARCHITECTURE AR

Software Architecture SWA

Design Software: 1 WB
We design an orthogonal, well-balanced and well-considered solution.

Software Architect

System Architecture SYA

Design Systems: 2 BB
We ensure that the solution fits optimally into its environment.

System Architect / Enterprise Architect

CONFIGURATION CF

Software Versioning VER

Version Artifacts: 1 BB
We place every artifact of the solution under strict version control.

Configuration Manager

Software Assembly ASM

Assemble Artifacts: 1 BB
We build and package the solution through an automated and repeatable mechanism.

Build Manager / Build Engineer

ANALYTICS AC

Software Reviewing REV

Review Code: 4 WB
We regularly and semantically peer-review the source code of the solution.

Software Tester

Software Testing TST

Test Solution: 2 BB
We adequately test the functional and non-functional aspects of the solution.

Software Tester

MANAGEMENT MG

Product Management PRD

Push Product: 3 BB
We continuously push the development and release of the solution to the users.

Product Manager / Product Owner

Project Management PRJ

Steer Process: 3 BB
We rigorously balance time, cost and scope to react on changes and reach the goals.

Project Manager

business-oriented & domain-specific | **constructive** & technological | **infrastructural** & technological | **analytical** & domain-specific | **people-oriented** & process-oriented

EXPERIENCE EX

User Experience UXP

Optimize Workflows: 3 BB
We align the solution to the perspective of the target audience.

User Experience Expert

User Interface UID

Design User Interfaces: 2 WB
We design a useful, intuitive, and beautiful user interface for the solution.

User Interface Designer / Graphics Designer

DEVELOPMENT DV

Software Development DEV

Implement Code: 1 WB
We develop the solution outside-in, from coarse to fine aspects.

Software Engineer / Software Developer

Software Refactoring REF

Refactor Code: 4 BB
We regularly and holistically refactor the solution to ensure long-term quality.

Software Engineer / Software Developer

DELIVERY DL

Software Deployment DPL

Deploy Artifacts: 1 BB
We ship and deploy the solution through an automated and repeatable mechanism.

System Engineer

System Operations OPS

Operate Solution: 4 BB
We ensure that our infrastructures and the solution can be operated in a resilient and secure way.

System Administrator / System Operator

COMPREHENSION CP

Usage Documentation DOC

Document Solution: 2 WB
We adequately document the usage and operation of the solution.

Technical Writer

User Training TRN

Train Users: 4 BB
We adequately train the users and operators of the solution.

Product Expert

ADJUSTMENT AD

Project Coaching COA

Support Members: 4 BB
We ensure that project members use state-of-the-art methodology, technology, and tools.

Project Coach / Methodology Master

Change Management CHG

Involve Stakeholders: 3 BB
We ensure that all stakeholders of the solution are suitably involved.

Change Manager

WB white-box view (details before whole) | BB black-box view (whole before details) | X scalability layer (from 4/most to 1/least dispensable)

EF 02.2

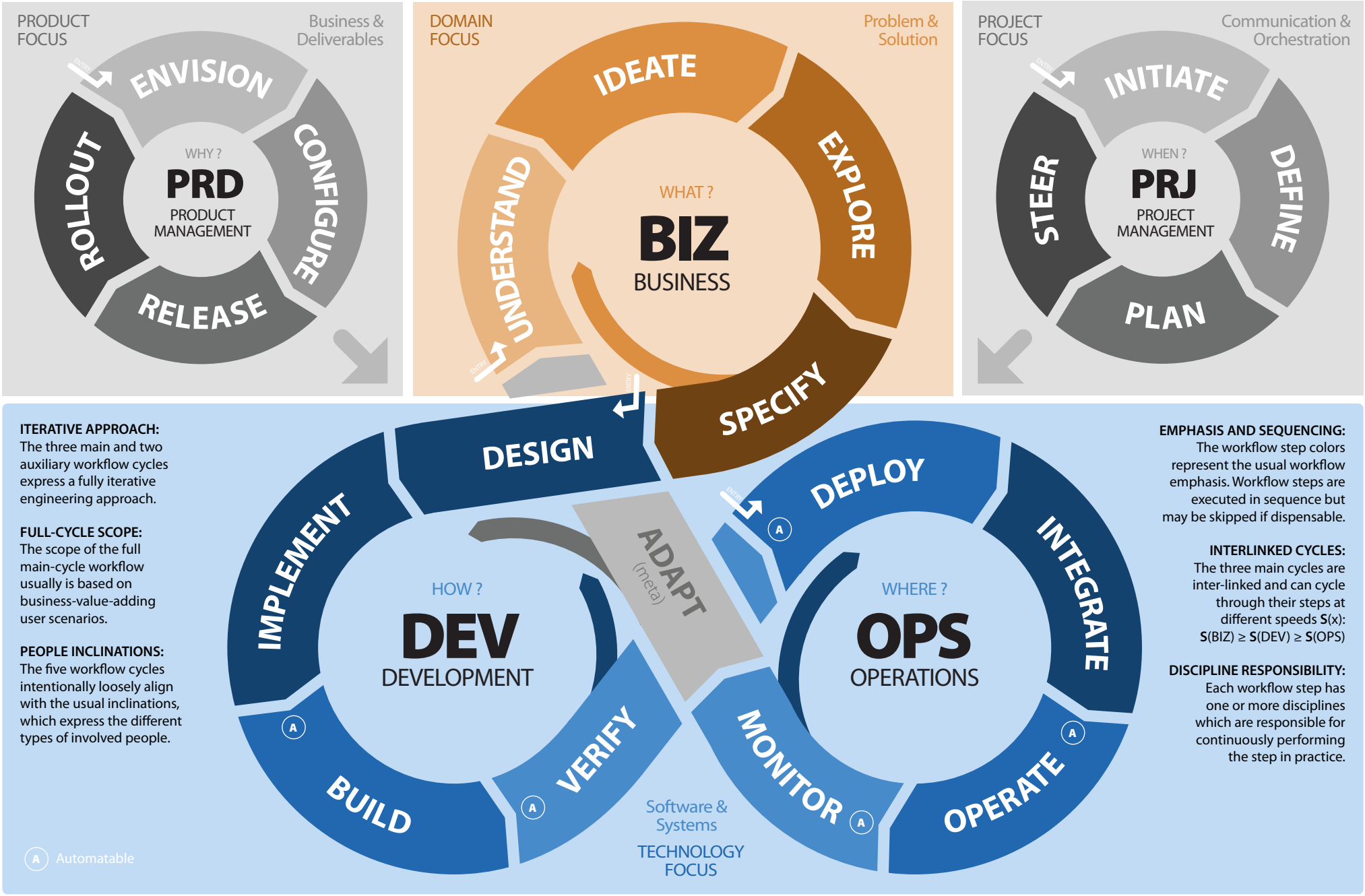
Intellectual Content: Version 3.0.9 (2022-12-10), Authored 2009-2022 by Dr. Ralf S. Engelschall and Rudolf Koster.
Graphical Illustration: Version 3.0.2 (2021-09-11), Copyright © 2009-2021 Dr. Ralf S. Engelschall <http://engelschall.com>. All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

inclination & knowledge

ENGINEERING FUNDAMENTALS

Software Engineering Workflow

TECHNISCHE UNIVERSITÄT MÜNCHEN



ITERATIVE APPROACH:
The three main and two auxiliary workflow cycles express a fully iterative engineering approach.

FULL-CYCLE SCOPE:
The scope of the full main-cycle workflow usually is based on business-value-adding user scenarios.

PEOPLE INCLINATIONS:
The five workflow cycles intentionally loosely align with the usual inclinations, which express the different types of involved people.

(A) Automatable

EMPHASIS AND SEQUENCING:
The workflow step colors represent the usual workflow emphasis. Workflow steps are executed in sequence but may be skipped if dispensable.

INTERLINKED CYCLES:
The three main cycles are inter-linked and can cycle through their steps at different speeds **S**(x):
S(BIZ) ≥ **S**(DEV) ≥ **S**(OPS)

DISCIPLINE RESPONSIBILITY:
Each workflow step has one or more disciplines which are responsible for continuously performing the step in practice.

Intellectual Content: Version 1.1.1 (2021-11-29), Authored 2019-2021 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.1.0 (2021-09-12), Copyright © 2020-2021 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

ENGINEERING FUNDAMENTALS

Software Engineering Steps

TECHNISCHE UNIVERSITÄT MÜNCHEN

EF 02.4

PRODUCT MANAGEMENT — PRD

ENVISION
PRD UXP

Envision Solution:
We envision the solution functionality and quality from the business and user perspectives.

CONFIGURE
PRD VER

Configure Version:
We configure versions of the solution from versioned artifacts and their feature-sets.

RELEASE
PRD ASM DOC

Release Version:
We create and release a distinct version of the solution.

ROLLOUT
CHG TRN

Rollout Version:
We adequately inform, involve and train the users and operators of the solution.

BUSINESS EFFORTS — BIZ

UNDERSTAND
REQ UXP PRD

Understand Problem:
We empathically understand the problem and requirements of the users.

IDEATE
UXP REQ

Ideate Solution:
We find an adequate solution for the problem and the requirements of the users.

EXPLORE
UXP UID SWA DEV

Explore Ideas:
We prototype, explore, and assess ideas, approaches and technologies for the solution.

SPECIFY
DOM UID

Specify Solution:
We rigorously and completely specify the functionality and quality of the solution.

DEVELOPMENT EFFORTS — DEV

DESIGN
SWA SYA

Design Architecture:
We design how to implement the solution in an orthogonal, adequate and sustainable way.

IMPLEMENT
DEV DOC REF REV

Implement Solution:
We implement the solution outside-in, from coarse to fine aspects.

BUILD
ASM VER

Build Artifacts:
We build and package the solution from versioned artifacts.

VERIFY
TST REV

Verify Solution:
We rigorously, but adequately, review and test the functional and non-functional aspects of the solution.

OPERATIONS EFFORTS — OPS

DEPLOY
DPL OPS SYA

Deploy Artifacts:
We ship and deploy the solution releases and their updates in an automated and repeatable way.

INTEGRATE
DPL OPS SYA

Integrate Environment:
We integrate the solution with its target environment.

OPERATE
OPS

Operate Solution:
We ensure that our infrastructures and the solution can be operated in a resilient and secure manner.

MONITOR
OPS TST

Monitor Solution:
We continuously monitor our infrastructures and the solution under run-time.

PROJECT MANAGEMENT — PRJ

INITIATE
PRJ COA

Initiate Project:
We initially setup the project on the contract and resource level.

DEFINE
PRJ PRD

Define Constraints:
We define the constraints of the project on the time, cost, and scope level.

PLAN
PRJ PRD

Plan Tasks:
We continuously plan the next iterations, their steps and their tasks in the project.

STEER
PRJ

Steer People:
We rigorously and continuously balance time, cost and scope to react on changes and still fulfilling the constraints.

steps & goals

Intellectual Content: Version 1.0.1 (2021-09-13), Authored 2021 by Dr. Ralf S. Engelschall and Rudolf Koster.
Graphical Illustration: Version 1.0.1 (2021-09-13), Copyright © 2021 Dr. Ralf S. Engelschall <http://engelschall.com>. All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

ENGINEERING FUNDAMENTALS

Software Engineering Process

TECHNISCHE UNIVERSITÄT MÜNCHEN

1. WORKFLOW CYCLES
The workflow has five cycles which continuously iterate through their steps. Workflow steps are executed in each cycle in sequence, but may be skipped if dispensable in a particular iteration of the process. The length of an iteration is arbitrary, but can be e.g. about 1/3 of a Scrum sprint.

2. WORKFLOW STEPS:
The workflow steps describe a logical activity which has to be performed. Each step relates to one or more discipline areas and their corresponding disciplines, which express the operative responsibilities for each workflow step. In each discipline individual roles act.

3. WORKFLOW ROLES:
The workflow roles are held by individual persons. Each role is primarily responsible for a particular workflow step. In addition, each role can be secondarily responsible for other workflow steps or at least actively support those steps.

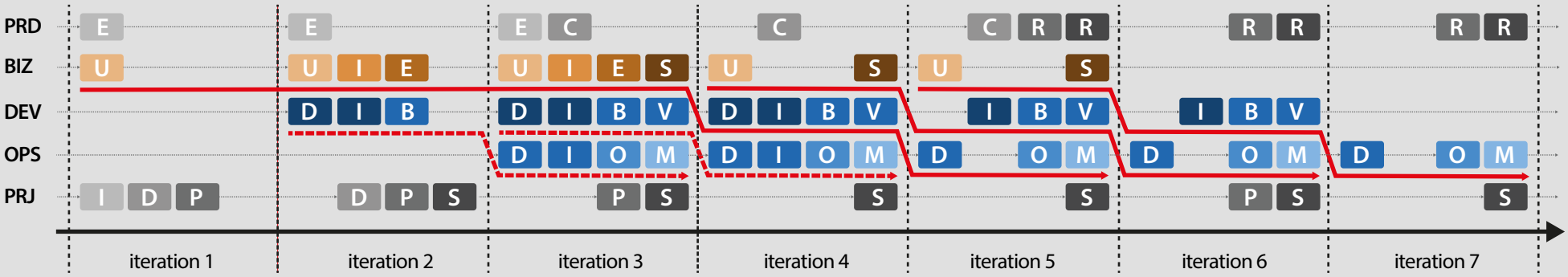
4. PROJECT SCHEDULE:
To create a particular project execution schedule, the five cycles, their iterations and their steps have to be mapped onto a timeline. The cycles are mapped onto (horizontal) timeline tracks, the iterations are mapped onto (vertical) timeline phases, and the steps are mapped onto timeline activities.

5. PROCESS FLOWS (THE CRUX):
The activities across the cycles can (and should) be linked into individual (diagonal) waterfall-like flows, although the execution schedule, from the perspective of the cycles, is fully iterative. There are multiple such flows in parallel and they are usually highly interleaved on the project timeline in order to maximally utilize the team.

6. PROCESS ADAPTION:
In the meta-step ADAPT, the process is adapted by choosing which workflow steps are required for the next iteration. The major input for this decision is the current solution state and the feedback on it by the customer.

		business-oriented & domain-specific				constructive & technological				infrastructural & technological				analytical & domain-specific				people-oriented & process-oriented			
		AN		EX		AR		DV		CF		DL		AC		CP		MG		AD	
		REQ	DOM	UXP	UID	SWA	SYA	DEV	REF	VER	ASM	DPL	OPS	REV	TST	DOC	TRN	PRD	PRJ	COA	CGH
		Requirements Engineer	Business Architect	User Experience Expert	User Interface Designer	Software Architect	System Architect	Software Developer	Software Developer	Configuration Manager	Build Manager	System Engineer	System Administrator	Software Tester	Software Tester	Technical Writer	Product Trainer	Product Owner	Project Manager	Project Coach	Change Manager
PRD	ENVISION	+	+	*														*			
	CONFIGURE	+	+	+						*								*			
	RELEASE									+	*					*	+	*	+		+
	ROLLOUT																*	+	+		*
BIZ	UNDERSTAND	*	*	*														*			
	IDEATE	+	+	*														+			
	EXPLORE	+	+	*	*	*	+	*										+			
	SPECIFY	*	*	+	*	+	+								+	+		+			
DEV	DESIGN	+	+	+	+	*	*			+	+	+		+	+			+			
	IMPLEMENT				*		+	*	*	+	+			*		*					
	BUILD					+	+	+		*	*	+		+							
	VERIFY	+	+	+	+	+	+	+						*	*			+			
OPS	DEPLOY					+	*	+		+	+	*	*								+
	INTEGRATE					+	*			+	+	*	*								+
	OPERATE											+	*								
	MONITOR	+	+	+		+	+					+	*		*			+			
	ADAPT	+	+	+		+	+											*	*	*	+
PRJ	INITIATE																		*	*	
	DEFINE	+	+			+												*	*		
	PLAN	+	+			+												*	*		
	STEER	+	+			+													*		

* responsible (primarily)
* responsible (secondarily)
\+ supporting



EF 02.5

Intellectual Content: Version 1.0.9 (2025-10-18), Authored 2019-2021 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.1.1 (2021-08-11), Copyright © 2020-2021 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

roles & tasks

ENGINEERING FUNDAMENTALS

Software Engineering Artifacts

TECHNISCHE UNIVERSITÄT MÜNCHEN

EF 02.6

1 Software **Requirements** Specification — *input / what* — REQ

Requirements: **Customer Journey** — REQ UXP PRD — 2 ENVISION	Domain Model: **Personas** — REQ DOM UXP — 3 UNDERSTAND
Requirements: **Solution Vision** — REQ UXP PRD — 2 ENVISION	Domain Model: **Test Cases** — REQ DOM TST — 3 SPECIFY
Requirements: **Functional Requirements** — PRD UXP REQ — 1 UNDERSTAND	User Interface: **Usage Concept** — UXP UID — 2 SPECIFY
Requirements: **Non-Functional Requirem.** — PRD SWA REQ — 1 UNDERSTAND	User Interface: **Language Conventions** — REQ UXP UID — 3 SPECIFY
Domain Model: **Data Model** — REQ DOM — 1 SPECIFY	User Interface: **Dialog Patterns** — UXP UID — 3 SPECIFY
Domain Model: **Use Cases** — UXP REQ DOM — 1 SPECIFY	User Interface: **Dialog Storyboard** — REQ UXP UID — 1 SPECIFY
Domain Model: **Use Case Scenarios** — UXP REQ DOM — 2 SPECIFY	User Interface: **Visual Design** — PRD UXP UID — 3 SPECIFY

2 Software **Architecture** Specification — *input / how* — ARC

Viewpoint: **Context View** — SYA SWA PRD — 2 ENVISION	Perspective: **Configurability & Extensibili.** — DEV SWA DPL — 3 INTEGRATE
Viewpoint: **Functionality View** — DOM SYA SWA — 1 DESIGN	Perspective: **Performance & Scalability** — OPS DPL SYA SWA — 1 DESIGN
Viewpoint: **Information View** — DOM SWA — 1 DESIGN	Perspective: **Availability & Recoverability** — SYA SWA DPL OPS — 2 OPERATE
Viewpoint: **Concurrency View** — DEV SYA SWA — 2 DESIGN	Perspective: **Reliability & Resilience** — DEV SYA SWA — 2 DESIGN
Viewpoint: **Development View** — ASM VER SWA DEV — 3 IMPLEMENT	Perspective: **Interoperability & Compatib.** — DEV SWA — 3 DESIGN
Viewpoint: **Deployment View** — SYA SWA OPS DPL — 1 DEPLOY	Perspective: **Compliance & Tracability** — PRD TST SWA DEV — 3 IMPLEMENT
Viewpoint: **Operations View** — SYA SWA DPL OPS — 2 OPERATE	Perspective: **Security & Safety** — DEV SWA — 1 DESIGN

4 Software **Documentation** Results — *output / what* — DOC

User Guide: **Usage Tutorial** — UXP TRN DOC — 2 SPECIFY	Operation Guide: **Configuration Reference** — TRN DOC DPL DEV — 1 IMPLEMENT
User Guide: **Functionality Reference** — DOM TRN DOC — 1 SPECIFY	Operation Guide: **Deployment Procedure** — TRN DOC OPS DPL — 1 DEPLOY
User Guide: **Release Information** — PRD VER — 3 RELEASE	Operation Guide: **Operation Procedures** — TRN DOC DPL OPS — 2 OPERATE

3 Software **Implementation** Results — *output / how* — IMP

Source Code: **Application** — REF DEV — 1 IMPLEMENT	Binary Code: **Application** — ASM — 1 BUILD
Source Code: **Build Automation** — DEV VER ASM — 2 BUILD	Source Code: **Deployment Automation** — DEV VER DPL — 3 DEPLOY
Source Code: **Test Automation** — DEV TST — 3 VERIFY	Source Code: **Operation Automation** — DEV OPS — 3 OPERATE

Notice: Artifact Tagging

Each Artifact is tagged with the primarily and secondarily responsible **Disciplines**, the primary **Step** of the **Workflow** where the Artifact is developed, and the **Scalability Layer** (from 3/most to 1/least dispensible).

Notice: Domain vs. Technology

The **Software Requirements Specification** and the **Software Documentation Results** primarily have a **domain-specific** focus. The **Software Architecture Specification** and the **Software Implementation Results** primary have a **technological** focus.

Notice: Artifacts vs. Aspects

The four **Artifact Sets** shown here just cluster the individual **Artifacts** and their contained **Aspects**. The **Artifacts** can be represented in an arbitrary graphical and/or textual form and be provided in an arbitrary format. The **Aspects** just structure an individual Artifact internally.

Notice: Internal vs. External

In a Software Engineering project, additional **internal Artifacts** are created by the **Disciplines** in order to perform their work efficiently and effectively. The Artifacts shown here are the **external** ones which glue together the Disciplines and which are part of the delivery set.

artifacts & deliverables

Intellectual Content: Version 1.0.1 (2026-1-1-1), Authored 2021-2026 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.1 (2026-1-1-1), Copyright © 2021-2026 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.